Financial Workflow Function-Calling LLM

Fine-tuning, optimization, and deployment write-up

This document covers what was built and why: which model was fine-tuned and how, which methods were compared, how the model was optimized and deployed, and what the actual evaluation and performance numbers came out to be. Code, training scripts, and everything needed to reproduce these runs is in the accompanying repository.

Choosing the base model

The base model is Qwen2.5-7B-Instruct. The main reason is that it already produces tool calls in the exact format this task needs - its chat template wraps function calls as <tool_call>{"name": ..., "arguments": ...}</tool_call> by default, because Qwen2.5 was itself trained on function-calling data. That matters more than it sounds: fine-tuning on ToolACE becomes a matter of sharpening a skill the model already has, rather than teaching it a brand new output format from nothing, which gets more out of the same amount of training data.

Size mattered too. The brief asks to optimize for accuracy first, then latency, then cost, in that order. A 7B model is close to the smallest size where function-calling accuracy doesn't fall off a cliff, and it's small enough that a single H100 can serve 16 to 32 concurrent requests comfortably once quantized. Going smaller would trade meaningful accuracy for latency gains that can be had more cheaply through quantization and serving optimizations instead. Going bigger would help accuracy a little further but actively work against the latency and cost priorities that come after it. Qwen2.5-7B-Instruct is also Apache-2.0 licensed, which avoids any licensing friction for a client's production deployment, and it's Apache-2.0-compatible with Llama-3.1-8B being the next closest alternative - slightly larger for slightly worse function-calling accuracy, with a more restrictive commercial license.

Fine-tuning: what was tried and what was picked

Three methods were actually run on the same data, not just discussed: LoRA, QLoRA, and a full fine-tune. All three used the same tokenization and loss-masking pipeline, where loss is computed only on the assistant's turns - the model never gets trained to predict the system prompt, the user's message, or a tool's response, only its own replies and function calls. That distinction is worth being explicit about, because with a multi-turn dataset like ToolACE (user, then a tool call, then a tool result, then a reply, and so on, repeated many times per conversation), getting this wrong quietly teaches the model to complete its own input instead of doing the actual task.

LoRA finished in just under two hours and reached a training loss of 0.173 and an evaluation loss of 0.176. QLoRA took about two hours and forty minutes and landed on essentially the same numbers - 0.174 and 0.177. The two methods produce almost identical quality, which is expected: LoRA and QLoRA fine-tune the same small set of adapter weights, the only difference is that QLoRA keeps the

frozen base model compressed to 4-bit and decompresses it on the fly during every forward and backward pass. That decompression is exactly why QLoRA took about 40% longer to train despite reaching the same result. Since an 80GB H100 has plenty of room to hold a 7B model in full precision, none of QLoRA's memory savings were actually needed here, so paying that speed cost bought nothing. LoRA is the better choice for this hardware.

The full fine-tune only ran for one epoch, with a much smaller learning rate than the LoRA runs, because updating every one of the model's 7.6 billion parameters at once needs a gentler step size and less exposure to a single, narrow-domain dataset to avoid overfitting or damaging the model's general instruction-following ability. It finished in 39 minutes and its training loss, 0.235, looks worse than LoRA's on paper - but that number is averaged across the one epoch it saw, most of which was still early in training, whereas LoRA and QLoRA's numbers are dominated by their third and final epoch. Its evaluation loss actually came out lowest of the three, at 0.150. None of that settles which model is genuinely better at function calling, though - evaluation loss on ToolACE only measures how likely the model is to predict the next token correctly, which is not the same thing as getting a function call exactly right. That's what BFCL is for.

LoRA is the method that got merged into a dense checkpoint and carried forward into quantization and deployment.

Why plain Trainer and PEFT instead of TRL

Training uses Hugging Face's transformers.Trainer directly with PEFT for the LoRA and QLoRA adapters, rather than TRL's SFTTrainer. TRL's completion-only data collator masks loss based on matching a single instruction and response template pair in the text, which works cleanly for simple single-turn chat data but doesn't generalize to a conversation with an arbitrary number of alternating roles the way ToolACE's data does. Instead, each conversation is tokenized turn by turn using the model's own chat template, and the token span each turn contributes is masked or kept based on whether that turn came from the assistant. It's a small amount of extra code, but it gives exact control over what the model is and isn't trained on, and it's easy to verify is correct rather than trusting a string-matching heuristic to hold up across an entire dataset.

Optimizing for production: quantization

Two quantization methods were produced from the merged checkpoint and both were actually deployed and evaluated, not just reasoned about on paper: FP8, using LLM Compressor's FP8_DYNAMIC scheme, and AWQ INT4. The FP8 checkpoint came out to 8.2GB and the AWQ checkpoint to 5.2GB.

FP8 keeps static per-channel weight scales computed once offline and dynamic per-token activation scales computed at inference time, which means no calibration dataset is needed and there's no risk of a calibration set that doesn't match real traffic. AWQ was calibrated on 256 of ToolACE's own held-out validation examples, rendered through the real chat template, so its calibration distribution matches

actual function-calling traffic rather than AutoAWQ's generic default calibration text.

Run against the real BFCL Python subset, FP8 scored 77.35% overall on the live categories and 74.54% on the non-live categories. AWQ came in close behind at 76.31% and 74.23%. On the concurrency benchmark, FP8 handled 26.5 requests per second at a concurrency of 16 and 57.6 at a concurrency of 32; AWQ managed 24.0 and 47.2 at the same two levels. FP8 wins on both accuracy and speed, and by a wider margin on speed than on accuracy. That's not a coincidence: 4-bit weights need to be decompressed before every matrix multiplication, the same kind of overhead QLoRA pays during training, and that decompression cost outweighs the smaller checkpoint size. FP8, by contrast, runs natively on the H100's Hopper tensor cores, so it gets a genuine compute speedup rather than just a memory saving. FP8 is what's actually served.

Why vLLM for hosting

vLLM was picked as the serving framework for a few concrete reasons. It exposes an OpenAI-compatible chat completions API out of the box, which matters because Nebius TokenFactory itself is built on vLLM - building the deployment this way now means it's close to a direct lift onto TokenFactory later rather than a rewrite. Its PagedAttention scheme manages the KV cache far more efficiently than a plain generate loop would, which is what actually determines how many concurrent requests a given amount of GPU memory can serve well. Continuous batching lets new requests join a running batch immediately at the token level instead of waiting behind whichever request happens to be slowest, which is the main thing that keeps time-to-first-token low under concurrent load. And it has first-class support for the exact quantization formats used here.

Two of vLLM's flags matter specifically for this workload. Prefix caching is turned on, and it earns its keep here: every request to this agent shares a large, identical block of tool definitions in its system prompt, since a FinTech agent is calling from the same fixed set of internal APIs on every request, not a constantly-changing one. Caching that shared prefix means the server isn't repeatedly re-processing the same tool schema text on every single call. Chunked prefill is also on, so a long prompt with many tool definitions in context can't block token generation for other requests waiting behind it.

BFCL evaluation, in detail

The official bfcl-eval harness was used, evaluated against the "python" test group, which is BFCL's own built-in grouping covering every Python-based category rather than a hand-picked subset. On the non-live categories, which use static tool documentation, the fine-tuned FP8 model scored 95.00% on simple calls, 95.50% on multiple calls, 90.50% on parallel calls, 80.50% on parallel-multiple calls, and 80.00% on correctly recognizing when none of the available tools apply. On the live categories, which are user-contributed and generally harder, it scored 79.07%, 77.21%, 75.00%, and 66.67% on the same four call types respectively, 72.40% on irrelevance detection, and 87.50% on relevance detection.

The parallel-multiple category is the clear weak point, in both the live and non-live sets, and it scored the

same for the AWQ checkpoint too, so it isn't quantization noise or a fluke of one run - it's a genuinely harder skill: picking out the right subset of calls to make when several plausible-looking tools are available at once. This is a well known difficult category across the BFCL leaderboard generally, not something specific to this fine-tune.

It's worth noting that BFCL's own combined leaderboard table reports a much lower overall accuracy number for this model. That number blends in test groups that were never run at all - multi-turn conversations, web search, and long-term memory - and appears to score an untested group as a flat zero rather than leaving it out of the average. None of those groups are part of the Python subset or part of what this agent needs to do, so the category-by-category numbers above are the ones that actually reflect this model's performance.

Serving performance

The concurrency benchmark sent 200 real requests, each carrying a genuine ToolACE prompt and tool schema rather than a synthetic fixed-length one, at both 16 and 32 concurrent requests against the FP8 deployment. At 16 concurrent requests, time-to-first-token averaged 124 milliseconds, with the slowest one percent of requests still finishing their first token within about half a second. End-to-end latency averaged 546 milliseconds. At 32 concurrent requests - the top of the range the brief asks this to handle - time to first token actually dropped to an average of 79 milliseconds, and end-to-end latency to 422 milliseconds, with throughput rising to about 4,300 tokens and 58 requests per second.

Latency improving as concurrency increased, rather than degrading the way it usually does, is worth explaining rather than leaving as a surprising number. The benchmark cycles through a bounded pool of real prompts that share the same tool schemas, and by the time the 32-concurrency run started, vLLM's prefix cache was already warm from the 16-concurrency run that ran just before it. That isn't a benchmarking artifact to be suspicious of - it's the actual shape of this agent's real traffic. A FinTech agent calling into the same fixed internal API surface on every single request is exactly the situation where prefix caching pays off in production, so this result is representative of how the deployed system will actually behave, not an inflated one.

What was hardest about this

ToolACE doesn't represent function calls as JSON. Assistant turns are written as a pseudo-Python call list, something like [GetBalance(account_id="X"), CheckLimit(id="X")], and the function names inside it can contain spaces and even their own parentheses, so a straightforward string split doesn't hold up. Parsing that correctly needed a small bracket-aware parser that tracks nesting depth rather than relying on the first or last parenthesis it finds.

A second, less obvious problem showed up only at the full dataset size. Storing each example's parsed messages and tool list as nested Python objects worked fine on a small sample, but broke once all 11,300 examples were converted at once, because Hugging Face's datasets library has to infer one consistent

column schema across every row, and different tool definitions have genuinely different shapes. The fix was to store those fields as plain JSON strings and parse them back out wherever they're actually used, which sidesteps schema inference entirely.

The biggest single source of friction, though, was dependency conflicts between vLLM, AutoAWQ, LLM Compressor, and BFCL's own install, all of which pin specific, sometimes mutually incompatible versions of shared packages like torch and torchvision. Installing BFCL's evaluation harness into the same environment as the running vLLM server silently pulled in a different, incompatible torchvision build and broke the server outright. Training, serving, and evaluation now run in three separate virtual environments for exactly this reason. BFCL also needed a couple of flags that aren't obvious from its own documentation to work correctly against an already-running external server instead of trying to launch its own: skipping its built-in server setup, pointing it at the local checkpoint's tokenizer with an absolute path since the evaluation script changes directories partway through, and explicitly allowing it to overwrite its own cached results after a configuration change, since it otherwise silently reuses stale output from a previous run.